Table des matières

Labo 10 OLTP — Index, plans d'exécution et optimisation 1
Le terrain, en une minute 1
Partie 1 — Mesurer, puis lire le plan 3
Partie 2 — Créer l'index et remesurer 4
Partie 3 — Casser la sargabilité exprès 5
Partie 4 — L'index composite et l'ordre des colonnes 8
Partie 5 — Ce que les index coûtent 9
Partie 6 — Quand le planificateur se trompe 11
Partie 7 — Sur l'entrepôt 12
Partie 8 — Ce que le contraste vous a montré 15
Nettoyage 16
Et ensuite ? 16

Labo 10 OLTP — Index, plans d'exécution et optimisation

Laboratoire du Cours 10, voie OLTP · PostgreSQL + AdventureWorks OLTP · Formation SQL avancé

Ce module n'a qu'un laboratoire, et il se fait sur PostgreSQL. La raison est simple : c'est le seul des deux terrains où l'on peut créer un index, mesurer, le supprimer, et recommencer. La dernière partie se fait quand même sur l'entrepôt, où l'on mesure ce qui y remplace l'index. Comptez une heure trois quarts.

Les deux voies — Ce labo n'a pas de jumeau, et le suffixe `_OLTP` de son nom dit sur quel moteur on travaille plutôt que d'annoncer une alternative. La partie 8 traverse quand même sur l'entrepôt : le contraste est la moitié de la leçon.

Pourquoi on fabrique nos propres tables — Le portage d'AdventureWorks arrive avec ses index de clé primaire et de clé étrangère, ce qui rend impossible toute mesure « avant et après ». On se construit donc un terrain nu, par copie, dont on contrôle chaque index. C'est aussi ce qu'on fait en vrai avant de toucher à une base de production.

Le terrain, en une minute

© 2026 iAlexMG. Tous droits réservés.

Table de départ	Ce qu'elle contient	Lignes
`sales.salesorderheader`	une ligne par commande	31 465
`sales.salesorderdetail`	une ligne par ligne de commande	121 317
`production.product`	le catalogue	504

On en tire deux tables de travail sans le moindre index.

```
CREATE SCHEMA IF NOT EXISTS labo10;
SET search_path TO labo10, public;

-- Une copie à plat, jointure déjà faite, 121 317 lignes.
CREATE TABLE labo10.ligne AS
SELECT d.salesorderdetailid AS ligne_id,
       h.salesorderid,
       h.orderdate::date    AS jour,
       h.customerid,
       d.productid,
       d.orderqty,
       d.unitprice,
       d.linetotal,
       p.name               AS produit
FROM   sales.salesorderheader AS h
JOIN   sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
JOIN   production.product     AS p ON p.productid    = d.productid;

-- La même chose répliquée dix fois, pour que les mesures soient nettes.
CREATE TABLE labo10.grosse AS
SELECT ROW_NUMBER() OVER ()  AS ligne_id,
       l.salesorderid, l.jour, l.customerid, l.productid,
       l.orderqty, l.unitprice, l.linetotal, l.produit
FROM   labo10.ligne AS l
CROSS  JOIN generate_series(1, 10) AS g;

VACUUM (ANALYZE) labo10.ligne;
VACUUM (ANALYZE) labo10.grosse;

SELECT COUNT(*) FROM labo10.grosse;
```

Attendu : 1 213 170 lignes. `ANALYZE` calcule les statistiques dont le planificateur a besoin (théorie, section 7). `VACUUM` marque en plus chaque page comme lisible par tous, dans la carte de visibilité, ce qui permet au parcours d'index seul de la partie 2 de ne jamais ouvrir la table. Le démon de maintenance le ferait de lui-

© 2026 iAlexMG. Tous droits réservés.

même, mais à un moment qui ne dépend pas de vous, environ une minute après la création de la table lors de la rédaction. Sans ce `VACUUM`, les plans des parties 2 et 3 changeraient selon qu'il est passé ou non. Vérifiez qu'aucun index n'existe :

```
SELECT indexname FROM pg_indexes WHERE schemaname = 'labo10';
```

Attendu : aucune ligne. Un `CREATE TABLE AS` ne crée ni clé primaire ni index, ce qui est exactement ce qu'on veut ici.

Astuce — Dans `psql`, tapez `\timing on` une fois pour obtenir le temps de chaque commande. Dans pgAdmin et DBeaver, le temps s'affiche déjà. Quel que soit l'outil, exécutez chaque mesure deux ou trois fois et retenez la meilleure : la première paie le chargement des pages en mémoire.

Partie 1 — Mesurer, puis lire le plan

1.1 La mesure de départ

Théorie : `Theorie_10_SQL.md`, sections 1 et 3.

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*) FROM labo10.grosse WHERE productid = 870;
```

Attendu : un parcours séquentiel de toute la table, et parallèle. Le nœud `Gather` réunit le travail de trois processus, deux auxiliaires et le principal, qui lisent chacun une part de la table sous `Parallel Seq Scan`. Notez trois nombres du plan.

- Le temps total, `Execution Time`.
- Les lignes gardées par le filtre. Le nœud affiche `rows=15626.67 loops=3` : une moyenne par processus, soit 46 880 lignes en tout.
- Les pages lues, sous `Buffers` : `shared hit` pour celles que PostgreSQL a trouvées dans sa mémoire, `read` pour celles qu'il a dû aller chercher. Leur somme fait 13 824, la table entière, et elle ne bouge pas d'une exécution à l'autre, alors que le partage entre les deux bouge.

Le filtre garde 46 880 lignes sur 1 213 170, et le moteur a pourtant lu toutes les pages de la table pour les trouver. C'est le gaspillage que l'index va supprimer.

Astuce — Depuis PostgreSQL 18, `EXPLAIN ANALYZE` affiche les pages lues même sans l'option `BUFFERS`. Le labo l'écrit quand même, pour que la commande dise ce qu'elle mesure.

1.2 Anatomie du plan

Théorie : `Theorie_10_SQL.md`, section 3.

Relancez sans `ANALYZE` :

```
EXPLAIN SELECT COUNT(*) FROM labo10.grosse WHERE productid = 870;
```


© 2026 iAlexMG. Tous droits réservés.

Vous ne voyez plus que des estimations. Comparez la ligne `rows=` de cette sortie avec le `rows=` réel de la précédente. Sur une table fraîchement analysée, les deux devraient être proches. Gardez ce réflexe : c'est le premier diagnostic à faire sur toute requête lente.

1.3 Un plan de jointure

Théorie : `Theorie_10_SQL.md`, section 3.

```
EXPLAIN (ANALYZE)
SELECT g.produit, SUM(l.orderqty)
FROM   labo10.grosse AS g
JOIN   labo10.ligne  AS l ON l.salesorderid = g.salesorderid
                         AND l.productid    = g.productid
GROUP  BY g.produit;
```

Chaque ligne de `grosse` retrouve ici la ligne de `ligne` dont elle est une copie : une commande ne porte jamais deux fois le même produit, vérifié, donc la paire commande et produit désigne une seule ligne. Repérez le type de jointure choisi. Sans index d'aucun côté, le moteur construit une table de hachage sur la plus petite table et balaie la plus grande : c'est le `Hash Join`. Retenez cette logique, elle est constante : le hachage quand les deux côtés sont gros et non indexés, la boucle imbriquée quand la table interne est indexée et la table externe petite (section 3 de `Theorie_10_SQL.md`).

Partie 2 — Créer l'index et remesurer

2.1 L'index le plus simple

Théorie : `Theorie_10_SQL.md`, section 2.

```
CREATE INDEX idx_grosse_produit ON labo10.grosse (productid);
ANALYZE labo10.grosse;

EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*) FROM labo10.grosse WHERE productid = 870;
```

Attendu : le parcours séquentiel a disparu, remplacé par un `Index Only Scan`. Pour compter, le moteur n'a besoin d'aucune colonne de la table : l'index lui suffit, et `Heap Fetches: 0` dit qu'il n'a pas ouvert la table une seule fois, grâce au `VACUUM` du terrain. Le temps chute d'un ordre de grandeur, et les pages lues davantage encore, de 13 824 à 43.

2.2 La sélectivité décide

Théorie : `Theorie_10_SQL.md`, sections 2 et 7.

Demandons maintenant une colonne que l'index ne contient pas, `linetotal` : le moteur devra aller la chercher dans la table pour chaque ligne retenue.

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Un produit rare.
EXPLAIN (ANALYZE) SELECT SUM(linetotal) FROM labo10.grosse WHERE productid = 897;

-- Une condition qui garde toute la table.
EXPLAIN (ANALYZE) SELECT SUM(linetotal) FROM labo10.grosse WHERE productid > 700;
```

Attendu : un parcours par l'index sur le premier, un `Bitmap Index Scan` suivi d'un `Bitmap Heap Scan`, pour 20 lignes et quelques pages ; un parcours séquentiel sur le second. Le bitmap rassemble d'abord les emplacements trouvés dans l'index, les trie, puis lit la table dans l'ordre physique, ce qui coûte moins qu'un aller-retour par ligne. Sur le second, tous les produits vendus ont un numéro au-dessus de 700, et la condition garde les 1 213 170 lignes. Le planificateur a raison : quand on veut toute la table, passer par l'index pour aller ensuite chercher chaque ligne coûte plus cher que la lire d'un bout à l'autre. Un index ignoré n'est pas toujours un index inutile, c'est parfois un planificateur qui a bien travaillé.

Piège — Refaites la seconde requête avec `COUNT(*)` : le plan passe à un `Index Only Scan` sur toute la plage, parce que compter n'oblige pas à ouvrir la table et que l'index pèse treize fois moins qu'elle. Ce n'est pas l'index qui a changé, c'est la question. La colonne demandée décide autant que le filtre.

2.3 Le parcours d'index seul

Théorie : `Theorie_10_SQL.md`, sections 2 et 5.

```sql
-- L'index porte productid ; la requête ne demande que productid.
EXPLAIN (ANALYZE) SELECT productid FROM labo10.grosse WHERE productid = 870;

-- La requête demande une colonne de plus.
EXPLAIN (ANALYZE) SELECT productid, linetotal FROM labo10.grosse WHERE productid = 870;
```

La première peut devenir un `Index Only Scan`, la seconde non, puisque `linetotal` n'est pas dans l'index. Ajoutez-la sans la mettre dans le tri :

```sql
CREATE INDEX idx_grosse_produit_couvrant
    ON labo10.grosse (productid) INCLUDE (linetotal, orderqty);
ANALYZE labo10.grosse;

EXPLAIN (ANALYZE) SELECT productid, linetotal FROM labo10.grosse WHERE productid = 870;
```

Attendu : `Index Only Scan`. La table n'est plus ouverte du tout.

Piège — Un `Index Only Scan` affiche parfois un `Heap Fetches` non nul, ce qui veut dire que le moteur a quand même dû ouvrir la table. La cause est la visibilité : PostgreSQL doit vérifier que la ligne est visible pour votre transaction, et il ne peut sauter cette vérification que sur les pages marquées propres par le nettoyage. Un `VACUUM labo10.grosse;` remet les compteurs à zéro. Le Cours 11 explique d'où vient cette contrainte.

Partie 3 — Casser la sargabilité exprès

© 2026 iAlexMG. Tous droits réservés.

C’est la partie la plus instructive du labo, parce que les trois requêtes qui suivent renvoient chacune le bon résultat.

3.1 Une fonction sur la colonne filtrée

Théorie : `Theorie_10_SQL.md`, section 4.

```sql
CREATE INDEX idx_grosse_jour ON labo10.grosse (jour);
ANALYZE labo10.grosse;

-- Non sargable.
EXPLAIN (ANALYZE)
SELECT SUM(linetotal) FROM labo10.grosse WHERE EXTRACT(YEAR FROM jour) = 2023;

-- Sargable, même résultat.
EXPLAIN (ANALYZE)
SELECT SUM(linetotal) FROM labo10.grosse WHERE jour >= '2023-01-01' AND jour < '2024-01-01';
```

Attendu : parcours séquentiel puis parcours d’index, et la même somme dans les deux cas, sur 208 170 lignes. Le premier plan porte la condition sous `Filter`, appliquée à chaque ligne lue ; le second sous `Index Cond`, la condition qui a servi à descendre l’index. L’index contient les dates, pas les années : le moteur ne peut pas deviner que la fonction est monotone.

Piège — Avec `COUNT(*)` à la place de `SUM(linetotal)`, les deux plans affichent un `Index Only Scan` sur `idx_grosse_jour`, et on croirait l’index servi deux fois. Il ne l’est qu’une fois. Pour la version non sargable, le moteur lit l’index d’un bout à l’autre comme une table plus mince, et filtre chaque entrée : 1 047 pages et une ligne `Filter`, mesuré, contre 181 pages et une ligne `Index Cond` pour l’intervalle. Le nom du nœud ne suffit pas ; c’est `Index Cond` qui dit que le moteur est descendu.

3.2 Un calcul sur la colonne filtrée

Théorie : `Theorie_10_SQL.md`, section 4.

```sql
CREATE INDEX idx_grosse_prix ON labo10.grosse (unitprice);
ANALYZE labo10.grosse;

EXPLAIN (ANALYZE) SELECT COUNT(*) FROM labo10.grosse WHERE unitprice * 1.15 > 1000;
EXPLAIN (ANALYZE) SELECT COUNT(*) FROM labo10.grosse WHERE unitprice > 1000 / 1.15;
```

Même résultat, deux plans. Le calcul doit vivre du côté de la constante.

3.3 Le début de chaîne, et la classe d’opérateurs

Théorie : `Theorie_10_SQL.md`, section 4.

```sql
CREATE INDEX idx_grosse_produit_nom ON labo10.grosse (produit);
ANALYZE labo10.grosse;
```


© 2026 iAlexMG. Tous droits réservés.

```sql
EXPLAIN (ANALYZE) SELECT SUM(linetotal) FROM labo10.grosse WHERE produit LIKE 'Road-150%';
EXPLAIN (ANALYZE) SELECT SUM(linetotal) FROM labo10.grosse WHERE produit LIKE '%150%';
```

La deuxième requête ne peut employer aucun index B-tree : un tri alphabétique ne sert à rien quand on cherche un fragment au milieu d'une chaîne. La première, elle, dépend d'un détail que la plupart des gens ignorent, et c'est pour cela qu'elle figure ici.

Un index B-tree sur du texte trie selon la collation de la base. Si cette collation n'est pas la collation binaire, son ordre ne correspond pas à celui que `LIKE` suppose, et le moteur refuse de s'en servir même pour un préfixe. Regardez ce que fait votre installation, puis créez l'index qui règle le cas :

```sql
SELECT datcollate FROM pg_database WHERE datname = current_database();

CREATE INDEX idx_grosse_produit_motif ON labo10.grosse (produit text_pattern_ops);
ANALYZE labo10.grosse;

EXPLAIN (ANALYZE) SELECT SUM(linetotal) FROM labo10.grosse WHERE produit LIKE 'Road-150%';
```

Si votre première mesure montrait déjà un parcours d'index, votre base est en collation binaire et le second index ne change rien. Si elle montrait un parcours séquentiel, comme sur la base de rédaction, en collation `French_Canada.1252`, vous venez de voir la différence : le nouveau plan descend `idx_grosse_produit_motif` par un `Index Cond` qui encadre le préfixe entre `Road-150` et `Road-151`, pour 20 900 lignes, et vous connaissez maintenant `text_pattern_ops`, qui est une des réponses les plus utiles qu'on puisse donner en entretien sur cette question.

3.4 Le `lower()` et l'index d'expression

Théorie : `Theorie_10_SQL.md`, section 4.

```sql
EXPLAIN (ANALYZE) SELECT SUM(linetotal) FROM labo10.grosse WHERE lower(produit) = 'road-150 red, 62';

CREATE INDEX idx_grosse_produit_minuscule ON labo10.grosse (lower(produit));
ANALYZE labo10.grosse;

EXPLAIN (ANALYZE) SELECT SUM(linetotal) FROM labo10.grosse WHERE lower(produit) = 'road-150 red, 62';
```

Parcours séquentiel, puis parcours d'index. Notez la contrainte que l'index d'expression impose : la requête doit écrire l'expression exactement comme l'index la déclare. Un `upper()` ailleurs dans le code ne profitera de rien.

Rappel — C'est la même mécanique qu'au Cours 08. L'index GIN servait `document @> '{"pays": "Canada"}'` et pas `document ->> 'pays' = 'Canada'`, alors que les deux requêtes renvoient la même chose. Vous avez maintenant le mot pour le dire.

3.5 La conversion sur la colonne, la plus sournoise

Théorie : `Theorie_10_SQL.md`, section 4.

© 2026 iAlexMG. Tous droits réservés.

Celle-ci naît d'une bonne intention, et c'est ce qui la rend fréquente.

```
CREATE TABLE labo10.client_texte AS
SELECT customerid::text AS cle, COUNT(*) AS commandes
FROM   labo10.grosse GROUP BY 1;
CREATE INDEX idx_client_texte ON labo10.client_texte (cle);
ANALYZE labo10.client_texte;

-- L'application envoie un nombre. PostgreSQL refuse net.
SELECT * FROM labo10.client_texte WHERE cle = 29825;
```

Attendu : une erreur, parce qu'aucun opérateur ne compare du texte à un entier. Le réflexe naturel est d'ajouter une conversion, et le réflexe naturel choisit le mauvais côté.

```
-- La correction qui fait taire l'erreur, et qui tue l'index.
EXPLAIN (ANALYZE) SELECT * FROM labo10.client_texte WHERE cle::int = 29825;

-- La correction qui garde l'index.
EXPLAIN (ANALYZE) SELECT * FROM labo10.client_texte WHERE cle = '29825';
```

Les deux renvoient la même ligne. La première convertit la colonne, donc lui applique une fonction, donc renonce à l'index ; la seconde convertit la constante, ce qui ne coûte rien. Le message d'erreur a disparu dans les deux cas, et c'est bien le problème : rien ne distingue la bonne correction de la mauvaise, sauf le plan.

Partie 4 — L'index composite et l'ordre des colonnes

Théorie : `Theorie_10_SQL.md`, section 5.

```
CREATE INDEX idx_grosse_client_jour ON labo10.grosse (customerid, jour);
ANALYZE labo10.grosse;

-- Sert : préfixe complet.
EXPLAIN (ANALYZE) SELECT COUNT(*) FROM labo10.grosse WHERE customerid = 29825;

-- Sert : les deux colonnes, dans l'ordre.
EXPLAIN (ANALYZE) SELECT COUNT(*) FROM labo10.grosse
WHERE  customerid = 29825 AND jour >= '2023-01-01';

-- On saute la première colonne : un seul jour.
EXPLAIN (ANALYZE) SELECT COUNT(*) FROM labo10.grosse WHERE jour = '2024-06-01';
```


© 2026 iAlexMG. Tous droits réservés.

Attendu : les deux premières requêtes descendent `idx_grosse_client_jour` une seule fois, ce que dit la ligne `Index Searches: 1`. La troisième ne peut pas y entrer d'un coup, et elle retombe sur l'index `idx_grosse_jour` de la partie 3, une descente elle aussi. Supprimez-le pour voir ce que devient la règle du préfixe :

```
DROP INDEX labo10.idx_grosse_jour;
EXPLAIN (ANALYZE) SELECT COUNT(*) FROM labo10.grosse WHERE jour = '2024-06-01';
```

Attendu : pas de parcours séquentiel. Le plan passe par `idx_grosse_client_jour`, avec `Index Searches: 247` et environ 1 500 pages lues, pour les mêmes 580 lignes. C'est le parcours par saut de PostgreSQL 18 : il ouvre l'index à chaque groupe de clients et y cherche le jour, comme on chercherait tous les Camille en ouvrant l'annuaire à chaque nom de famille. Il bat encore le parcours séquentiel, qui lirait les 13 824 pages de la table, mais il lit plus de cent fois les pages d'un index dont `jour` est la tête.

Vérifiez enfin l'ordre inverse. Pour comparer les deux index sur la même requête, on retire le bon le temps d'une transaction : PostgreSQL annule aussi un `DROP INDEX`, comme tout le DDL (Cours 01, section 2), et l'index revient au `ROLLBACK`.

```
CREATE INDEX idx_grosse_jour_client ON labo10.grosse (jour, customerid);
ANALYZE labo10.grosse;

BEGIN;
DROP INDEX labo10.idx_grosse_client_jour;
EXPLAIN (ANALYZE) SELECT COUNT(*) FROM labo10.grosse
WHERE  customerid = 29825 AND jour >= '2023-01-01';
ROLLBACK;
```

Attendu : 3 210 lignes comme plus haut, mais `Index Searches: 398` et environ 1 300 pages, contre une descente et six pages d'index sur `idx_grosse_client_jour`. Les deux index couvrent la même paire de colonnes, et ils ne se valent pas. Celui qui met l'égalité en premier et la plage en second est le bon : la plage placée devant disperse le tri, et la colonne suivante ne sert plus qu'à sauter, au plus une descente par jour de la plage.

Partie 5 — Ce que les index coûtent

5.1 L'espace

Théorie : `Theorie_10_SQL.md`, section 6.

```
SELECT pg_size_pretty(pg_relation_size('labo10.grosse'))        AS table_seule,
       pg_size_pretty(pg_indexes_size('labo10.grosse'))         AS tous_les_index,
       pg_size_pretty(pg_total_relation_size('labo10.grosse'))  AS total;

SELECT indexrelname, pg_size_pretty(pg_relation_size(indexrelid)) AS taille
FROM   pg_stat_user_indexes WHERE schemaname = 'labo10'
ORDER  BY pg_relation_size(indexrelid) DESC;
```


© 2026 iAlexMG. Tous droits réservés.

`pg_relation_size` mesure la table seule, `pg_indexes_size` ses index, `pg_total_relation_size` le tout, et `pg_size_pretty` écrit le résultat en ko ou en Mo. À ce stade du labo, les index pèsent une fraction importante de la table, parfois davantage. C'est le loyer de la section 2 de la théorie, en chiffres.

5.2 Les écritures

Théorie : `Theorie_10_SQL.md`, section 6. Théorie : `Theorie_03_SQL.md`, sections 2 et 4.

```
-- Sur une copie sans index.
CREATE TABLE labo10.cible_nue AS SELECT * FROM labo10.ligne WITH NO DATA;
INSERT INTO labo10.cible_nue SELECT * FROM labo10.ligne;

-- Sur une copie couverte d'index.
CREATE TABLE labo10.cible_indexee AS SELECT * FROM labo10.ligne WITH NO DATA;
CREATE INDEX ON labo10.cible_indexee (productid);
CREATE INDEX ON labo10.cible_indexee (customerid, jour);
CREATE INDEX ON labo10.cible_indexee (produit);
CREATE INDEX ON labo10.cible_indexee (unitprice);
INSERT INTO labo10.cible_indexee SELECT * FROM labo10.ligne;
```

Comparez les deux temps d'insertion. L'écart est le prix que vous paierez à chaque écriture, pour toujours, en échange des gains mesurés aux parties 2 à 4. C'est cette comparaison qui transforme « ajoutons un index » en décision plutôt qu'en réflexe.

5.3 Les index qui ne servent jamais

Théorie : `Theorie_10_SQL.md`, section 6.

```
SELECT relname, indexrelname, idx_scan
FROM   pg_stat_user_indexes
WHERE  schemaname = 'labo10'
ORDER  BY idx_scan;
```

Les compteurs sont ceux de votre session de labo. En production, la même requête après quelques semaines d'exploitation désigne sans discussion les index qu'on paie sans les lire. C'est la première requête à écrire quand on hérite d'une base dont personne ne connaît l'histoire.

5.4 L'index partiel, pour une colonne peu sélective

Théorie : `Theorie_10_SQL.md`, sections 2 et 6. Théorie : `Theorie_03_SQL.md`, section 7.

```
-- Une colonne à deux valeurs, dont une seule vise une ligne sur cent : indexer tout n'a aucun intérêt.
ALTER TABLE labo10.grosse ADD COLUMN statut text;
UPDATE labo10.grosse SET statut = CASE WHEN ligne_id % 100 = 0 THEN 'a_verifier' ELSE 'ok' END;
ANALYZE labo10.grosse;

CREATE INDEX idx_grosse_a_verifier ON labo10.grosse (ligne_id) WHERE statut = 'a_verifier';
```


© 2026 iAlexMG. Tous droits réservés.

```sql
ANALYZE labo10.grosse;

EXPLAIN (ANALYZE) SELECT COUNT(*) FROM labo10.grosse WHERE statut = 'a_verifier';
SELECT pg_size_pretty(pg_relation_size('labo10.idx_grosse_a_verifier')) AS taille_index_partiel;
```

L'index ne contient qu'un pour cent des lignes, il est minuscule, et il répond exactement à la question qu'on pose vraiment. C'est la bonne réponse au cas de la colonne peu sélective.

Partie 6 — Quand le planificateur se trompe

6.1 Le chargement massif sans `ANALYZE`

Théorie : `Theorie_10_SQL.md`, section 7.

La table neuve est soustraite au démon de maintenance, qui lancerait sinon `ANALYZE` à sa place pendant que vous lisez, et effacerait ce qu'on veut voir.

```sql
CREATE TABLE labo10.fraiche (LIKE labo10.grosse) WITH (autovacuum_enabled = false);
INSERT INTO labo10.fraiche SELECT * FROM labo10.grosse;

-- Sans ANALYZE : la taille de la table, puis deux produits.
EXPLAIN SELECT * FROM labo10.fraiche;
EXPLAIN SELECT COUNT(*) FROM labo10.fraiche WHERE productid = 870;
EXPLAIN SELECT COUNT(*) FROM labo10.fraiche WHERE productid = 897;

ANALYZE labo10.fraiche;

-- Après ANALYZE.
EXPLAIN SELECT * FROM labo10.fraiche;
EXPLAIN SELECT COUNT(*) FROM labo10.fraiche WHERE productid = 870;
EXPLAIN SELECT COUNT(*) FROM labo10.fraiche WHERE productid = 897;
```

Attendu, avant `ANALYZE`. La table n'est pas vue vide : le premier plan annonce environ 634 000 lignes, faux de près de moitié mais tiré de la taille du fichier, et qui bouge de quelques dizaines d'une fois à l'autre avec le nombre exact de pages. Les deux produits reçoivent ensuite la même estimation, 1 322 à la rédaction, alors que le 870 compte 46 880 lignes et le 897 en compte 20 : sans statistiques de colonne, le planificateur applique à toute égalité la même part par défaut. Les plans des deux produits sont parallèles, et le `rows=` de leur nœud `Parallel Seq Scan` est l'estimation d'un seul processus, comme les 15 626,67 lignes par processus de la sous-partie 1.1.

Après `ANALYZE`, la taille est juste, 1 213 170, et le produit 870 passe à près de 20 000 lignes par processus, du même ordre que le réel. Le 897 reste estimé à plusieurs centaines de lignes par processus, pour 20 en tout : trop rare pour figurer parmi les cent valeurs fréquentes que garde l'échantillon, il reçoit une part

© 2026 iAlexMG. Tous droits réservés.

moyenne. C'est la troisième situation de la section 7 de la théorie, la distribution irrégulière. Les estimations d'après `ANALYZE` varient un peu d'une fois à l'autre, parce que l'échantillon est tiré au hasard.

Ce qui manquait avant `ANALYZE` n'était donc pas la taille de la table, c'était la répartition de ses valeurs. C'est la raison pour laquelle tout script d'alimentation devrait finir par un `ANALYZE`.

6.2 Les colonnes corrélées

Théorie : `Theorie_10_SQL.md`, section 7.

```
-- Deux colonnes parfaitement corrélées : le produit détermine son nom.
EXPLAIN (ANALYZE)
SELECT COUNT(*) FROM labo10.grosse
WHERE  productid = 870 AND produit = 'Water Bottle - 30 oz.';
```

Comparez l'estimation et le réel. Le planificateur suppose les deux conditions indépendantes, donc il multiplie leurs sélectivités et sous-estime lourdement. Dites-lui la vérité :

```
CREATE STATISTICS st_grosse_produit (dependencies)
    ON productid, produit FROM labo10.grosse;
ANALYZE labo10.grosse;

EXPLAIN (ANALYZE)
SELECT COUNT(*) FROM labo10.grosse
WHERE  productid = 870 AND produit = 'Water Bottle - 30 oz.';
```

L'estimation se rapproche du réel. Cette déclaration de statistiques étendues est peu connue, et elle règle une famille entière de plans absurdes.

6.3 Forcer un plan pour comprendre, jamais pour corriger

Théorie : `Theorie_10_SQL.md`, sections 3 et 7.

```
SET enable_seqscan = off;
EXPLAIN (ANALYZE) SELECT COUNT(*) FROM labo10.grosse WHERE productid > 700;
RESET enable_seqscan;
```

Le moteur emploie maintenant l'index qu'il refusait, et le temps est probablement pire. Ces interrupteurs servent à vérifier une hypothèse pendant un diagnostic, jamais à régler un problème en production : ils masquent la cause au lieu de la traiter.

Partie 7 — Sur l'entrepôt

© 2026 iAlexMG. Tous droits réservés.

Passez sur l'entrepôt, dans la ligne de commande DuckDB. Il n'y a aucun index à créer et aucun à maintenir, mais il y a beaucoup à voir, et tout se mesure.

```
USE dbo;
```

7.1 Deux tables, même contenu, deux rangements

Théorie : `Theorie_10_SQL.md`, section 8.

On fabrique la même table deux fois, une fois écrite dans l'ordre des dates, une fois mêlée. Le contenu est identique au sens de SQL : seules les lignes ne sont pas au même endroit du fichier.

```
-- Bien rangee par date.
CREATE OR REPLACE TABLE LABO10_TRIEE AS
SELECT f.OrderDateKey, d.FullDateAlternateKey AS OrderDate,
       f.CustomerKey, f.ProductKey, f.SalesAmount, g.n
FROM   dbo.FactInternetSales AS f
JOIN   dbo.DimDate AS d ON d.DateKey = f.OrderDateKey,
       (SELECT UNNEST(range(1, 41)) AS n) AS g
ORDER  BY d.FullDateAlternateKey;

-- Mal rangee.
CREATE OR REPLACE TABLE LABO10_MELEE AS
SELECT OrderDateKey, OrderDate, CustomerKey, ProductKey, SalesAmount, n
FROM   LABO10_TRIEE
ORDER  BY RANDOM();

SELECT COUNT(*) FROM LABO10_TRIEE;
```

Attendu : 2 415 920 lignes de chaque côté, et moins d'une seconde pour construire les deux. La jointure vers `DimDate` sert à obtenir une vraie date plutôt que la clé entière de la table de faits ; le `range(1, 41)` multiplie chaque vente par quarante, pour que la table dépasse largement la taille d'un bloc de lignes.

Avant d'attendre un élagage, comptez les blocs et regardez leurs bornes : une table d'un seul bloc n'a rien à élaguer. `PRAGMA_STORAGE_INFO` décrit le rangement d'une table, bloc par bloc et colonne par colonne.

```
SELECT COUNT(DISTINCT row_group_id) AS blocs
FROM   PRAGMA_STORAGE_INFO('LABO10_TRIEE');

SELECT row_group_id, stats
FROM   PRAGMA_STORAGE_INFO('LABO10_TRIEE')
WHERE  column_name = 'OrderDate' AND segment_type = 'DATE'
ORDER  BY row_group_id
LIMIT  3;
```


© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT row_group_id, stats
FROM   PRAGMA_STORAGE_INFO('LABO10_MELEE')
WHERE  column_name = 'OrderDate' AND segment_type = 'DATE'
ORDER  BY row_group_id
LIMIT  3;
```

Attendu : 20 blocs, et la même chose pour la table mêlée. Sur la table triée, chaque bloc porte quelques semaines ou quelques mois de dates : le bloc 1 va du 17 avril 2012 au 12 janvier 2013, le bloc 2 du 12 janvier au 16 février 2013, et janvier 2013 ne tombe que dans ces deux-là. Sur la table mêlée, chaque bloc va du 29 décembre 2010 au 28 janvier 2014, toute la période. La même requête sur `FactInternetSales` rend un seul bloc, pour ses 60 398 lignes : c'est pour cela que le labo en fabrique une table quarante fois plus grosse.

7.2 L'élagage, mesuré

Théorie : `Theorie_10_SQL.md`, section 8.

```sql
EXPLAIN ANALYZE
SELECT COUNT(*), SUM(SalesAmount) FROM LABO10_TRIEE
WHERE  OrderDate >= DATE '2013-01-01' AND OrderDate < DATE '2013-02-01';

EXPLAIN ANALYZE
SELECT COUNT(*), SUM(SalesAmount) FROM LABO10_MELEE
WHERE  OrderDate >= DATE '2013-01-01' AND OrderDate < DATE '2013-02-01';
```

Lisez le `Total Time` en tête de chaque plan, et relancez deux ou trois fois pour vous assurer que l'écart n'est pas du bruit. Attendu : la table mêlée met environ quatre fois le temps de la table triée, pour exactement les mêmes 66 480 lignes rendues et la même somme. Sur la machine de rédaction, 3,7 ms contre 15,1 ms, très stables d'une exécution à l'autre ; les valeurs absolues dépendront de la vôtre, c'est le rapport qui compte.

Sur la table triée, les blocs de lignes portent des dates voisines : le moteur lit leurs bornes, constate que la plupart n'ont aucune date de janvier 2013, et ne les ouvre pas. Sur la table mêlée, chaque bloc contient un peu de tout, donc aucune borne n'exclut quoi que ce soit, et il les lit tous.

C'est l'équivalent exact du parcours d'index contre le parcours séquentiel des parties précédentes, obtenu sans index et sans aucune commande, par le seul rangement physique des données. Retenez la conséquence pratique : sur un entrepôt, l'ordre dans lequel vous écrivez une table est une décision de performance, et personne ne vous la rappellera.

7.3 La sargabilité vaut aussi ici

Théorie : `Theorie_10_SQL.md`, sections 4 et 8. Théorie : `Theorie_02_SQL.md`, section 6.

```sql
EXPLAIN ANALYZE SELECT COUNT(*) FROM LABO10_TRIEE
WHERE  YEAR(OrderDate) = 2013 AND MONTH(OrderDate) = 1;

EXPLAIN ANALYZE SELECT COUNT(*) FROM LABO10_TRIEE
WHERE  OrderDate >= DATE '2013-01-01' AND OrderDate < DATE '2013-02-01';
```


© 2026 iAlexMG. Tous droits réservés.

Attendu : le même compte, 66 480, et environ deux fois plus de temps pour la version qui applique des fonctions, 3,0 ms contre 1,3 ms sur la machine de rédaction. Les bornes conservées par le moteur portent sur la valeur de `OrderDate`, pas sur le résultat de `YEAR(OrderDate)` : il ne peut donc éliminer aucun bloc et doit calculer la fonction sur chaque ligne avant de décider.

C'est la seule notion du module qui se transpose telle quelle d'un moteur à l'autre, et c'est pourquoi la section 4 vaut des deux côtés. Un filtre non sargable coûte des deux côtés, pour deux raisons différentes qui reviennent au même : il empêche le moteur d'écarter des données sans les regarder.

7.4 Nettoyage de l'entrepôt

```
DROP TABLE IF EXISTS LABO10_TRIEE;
DROP TABLE IF EXISTS LABO10_MELEE;
```

Astuce — Ces deux tables pèsent quelques dizaines de mégaoctets dans le fichier de l'entrepôt. Le `DROP` les retire du catalogue, mais le fichier ne rétrécit pas tout seul : c'est le comportement ordinaire d'un moteur qui réutilise l'espace libéré. Rien à faire, sauf si la place manque.

Partie 8 — Ce que le contraste vous a montré

Théorie : `Theorie_10_SQL.md`, sections 8 et 9.

Sujet	PostgreSQL, ce labo	L'entrepôt, partie 7
Accélérer un filtre	créer un index, mesurer, décider	rien à créer, l'élagage est automatique
L'outil de diagnostic	`EXPLAIN (ANALYZE, BUFFERS)`	`EXPLAIN ANALYZE` et son `Total Time`
Le nœud révélateur	`Seq Scan` contre `Index Scan`	pas de nœud : le même plan, quatre fois le temps
Le levier de rangement	l'ordre physique compte peu, l'index fait le travail	l'ordre physique fait tout, environ quatre fois le temps
Le coût du levier	l'espace et chaque écriture	réécrire la table pour la ranger autrement
Les statistiques	à rafraîchir, `ANALYZE` après tout chargement	tenues par le moteur, aucune commande
Repérer l'inutile	`pg_stat_user_indexes`, `idx_scan` à zéro	une table rangée sur une colonne jamais filtrée
La sargabilité	casse le parcours d'index	casse l'élagage, environ deux fois le temps

La troisième ligne mérite un mot, parce qu'elle est la plus déroutante du tableau. Sur PostgreSQL, le plan vous dit ce qui s'est passé : vous y lisez `Seq Scan` ou `Index Scan`, et vous savez. Sur l'entrepôt, le plan des deux requêtes est le même, au mot près, et seul le temps diffère. L'élagage ne se voit pas dans la forme du

© 2026 iAlexMG. Tous droits réservés.

plan, il se voit dans sa durée. C'est plus difficile à diagnostiquer, et c'est une raison de plus de mesurer plusieurs fois plutôt que de conclure d'une exécution. Les grandes plateformes rendent cela visible, Snowflake affichant les partitions parcourues sur le total dans son *Query Profile* d'après sa documentation.

La leçon de fond tient en une phrase. Sur le transactionnel, on optimise en ajoutant une structure ; sur l'entrepôt, on optimise en rangeant les données et en écrivant la requête autrement. Les deux mondes partagent une seule règle, et c'est celle de la sargabilité : ne touchez pas à la colonne que vous filtrez.

Nettoyage

```
DROP SCHEMA labo10 CASCADE;
```

Et ensuite ?

Vous savez maintenant mesurer plutôt que deviner, lire un plan, et décider d'un index sur autre chose qu'une intuition. Vous avez aussi croisé deux fois une notion qui n'a pas encore été expliquée : le `Heap Fetches` d'un parcours d'index seul, et la nécessité de nettoyer la table pour l'éviter.

Les deux viennent du même endroit. PostgreSQL ne modifie jamais une ligne sur place : il en crée une nouvelle version et laisse l'ancienne visible pour les transactions qui l'ont déjà vue. C'est le sujet du Cours 11, avec les transactions, l'isolation et la concurrence.

Prochaine étape : les exercices, dans `Exercices_corriges_10_SQL.md`.

© 2026 iAlexMG. Tous droits réservés.